

경력기술서

웹앱 프론트엔드 중심 | Enterprise Collaboration Platform

운영 서비스 경험, 기술 의사결정, 유지보수성 중심으로 정리

1. 프로젝트 개요

프로젝트	Enterprise Collaboration Platform
포지션	웹앱 프론트엔드 중심
핵심 주제	운영 중 변경되는 권한 정책을 안정적으로 반영하기 위한 권한 관리 구조 개선
연관 기능	조직도(Tree), 댓글, 메신저(WebSocket), 화상회의, AI Chat, OCR, STT, WebView Bridge, Camera, Permission

프로젝트 설명 기업용 협업 플랫폼에서 사용자 역할(Admin, Editor, Viewer)에 따라 접근 가능한 메뉴, 액션 버튼, 데이터 조회 범위가 달라지는 구조를 운영했다. 권한은 단순 메뉴 노출 여부를 넘어 상세 페이지 진입, 액션 실행, 조회 범위 제어까지 영향을 주었기 때문에, 화면마다 다른 방식으로 구현하면 운영 단계에서 빠르게 균열이 생기는 영역이었다.

핵심 문제 초기 구조에서는 각 화면이 직접 권한을 판단했다. 그 결과 동일한 권한 정책이 여러 컴포넌트와 화면에 분산되었고, 정책이 바뀔 때마다 수정 범위와 검증 범위가 함께 커졌다.

2. 담당 역할과 실무 범위

- 역할별 권한 구조를 프론트엔드 관점에서 재정의하고, 화면 단위로 흩어진 권한 조건을 공통 로직으로 수렴했다.
- 권한 정책 변경 시 수정 누락이 발생하지 않도록 Custom Hook 기반의 재사용 구조로 리팩토링했다.
- 운영 단계에서 발견된 권한 노출 이슈와 QA 회귀 버그를 분석하고, UI 노출 조건과 액션 가능 여부를 같은 기준으로 정리했다.
- 기획, QA, 연관 기능 개발자와 함께 역할별 동작 기준을 맞추며 권한 정책의 적용 범위를 명확히 했다.

3. 운영 경험 중심 상세 기술

3-1. 운영 중 실제로 마주한 문제

- 운영 과정에서 권한 정책이 여러 차례 변경되었다. 특정 역할이 볼 수 있는 메뉴는 유지하되 액션만 제한해야 하는 경우, 반대로 메뉴 진입 자체를 막아야 하는 경우가 반복적으로 발생했다.
- 초기 구현은 화면마다 조건문이 달라 동일한 역할이라도 목록 화면, 상세 화면, 모달, 버튼 영역에서 노출 기준이 일치하지 않는 문제가 있었다.
- QA에서는 권한이 없는 계정에서 일부 화면의 버튼이 노출되거나, 특정 메뉴는 숨겨졌지만 다른 진입 경로에서는 접근 가능한 버그가 반복적으로 발견되었다.
- 운영 관점에서도 권한 정책 변경이 생길 때마다 프론트엔드가 수정해야 할 화면 범위를 일일이 다시 추적해야 했고, 변경 영향도를 빠르게 설명하기 어려웠다.
- 유지보수하면서 가장 크게 느낀 문제는 기능 구현 속도보다 변경 비용이 더 빠르게 증가한다는 점이었다. 신규 기능을 추가할수록 권한 조건도 함께 복제되었고, 이후 정책이 바뀌면 복제된 조건이 다시 부채가 되었다.

3-2. 리팩토링을 결정한 이유

배경 권한 로직이 여러 화면에 분산된 상태에서는 기능 하나를 수정해도 실제 영향 범위를 끝까지 확신하기 어려웠다. 특히 운영 중 정책 변경은 기능 추가보다 더 자주 발생했고, QA가 같은 유형의 버그를 반복적으로 잡아내는 상황이 이어졌다.

의사결정 단순히 중복 코드를 줄이는 수준이 아니라, 권한 정책의 변경 지점을 한 곳으로 모으고 화면은 그 결과만 소비하도록 구조를 바꾸는 것이 우선이라고 판단했다. 즉, 구현 편의보다 운영 안정성과 회귀 방지에 초점을 두고 리팩토링을 진행했다.

협업 과정 기획과는 역할별로 어떤 기능이 '보여야 하는지', '실행 가능해야 하는지', '진입 자체를 막아야 하는지'를 구분해 기준을 정리했다. QA와는 테스트 시나리오를 화면 기준이 아니라 역할 기준으로 재정리해, 정책 변경 시 어디를 우선 검증해야 하는지 공통 언어를 맞췄다.

4. 가장 어려웠던 버그

권한 로직 분산으로 인해 일부 화면에서 권한 없는 액션이 노출되던 문제

Issue 권한 정책이 변경된 이후에도 일부 화면이 이전 조건을 그대로 사용하면서, 권한이 없는 사용자에게 특정 액션 버튼 또는 진입 경로가 남아 있는 문제가 발생했다.

Symptoms 사용자 입장에서는 어떤 화면에서는 버튼이 보이지 않는데, 다른 화면에서는 같은 기능 버튼이 노출되는 식으로 경험이 일관되지 않았다. QA 기준으로는 동일 역할 계정의 동작이 화면마다 달라 회귀 테스트 범위가 계속 늘어나는 현상이 나타났다.

Cause 권한 판단이 공통 레이어에 있지 않고 UI 컴포넌트마다 분산되어 있었기 때문이다. 동일한 역할이라도 각 화면이 서로 다른 조건문을 사용했고, 정책 변경 시 모든 화면이 동시에 수정되지 않으면 즉시 불일치가 발생하는 구조였다.

Debugging 문제가 화면 한 곳의 버그처럼 보였지만 실제로는 정책 반영 누락이 누적된 구조적 문제라고 판단했다. 그래서 단일 화면만 보지 않고, 역할별로 어떤 메뉴·버튼·조회 범위가 열려 있는지 전체 흐름을 다시 비교했다. 그 과정에서 동일 권한을 판단하는 코드가 여러 컴포넌트에 중복되어 있고, 화면마다 조건식의 기준이 조금씩 다르다는 점을 확인했다.

Solution 권한 판단을 화면 밖으로 이동시키고 Custom Hook으로 분리했다. 화면은 '직접 판단'하지 않고 Hook이 반환하는 canView, canEdit, canAccess 같은 결과를 사용하도록 바꿨다. 동시에 역할별 권한 매핑을 Hook 내부에서 관리해 정책 변경 시 수정 지점을 한 곳으로 제한했다.

Trade-off Context API로 전역 권한 상태를 공유하는 방식도 고려했지만, 이번 문제의 본질은 상태 공유보다 판단 로직의 일관성과 재사용성이었다. HOC는 선언적으로 권한을 감쌀 수 있다는 장점이 있었지만, 권한 종류가 늘어날수록 Wrapper 계층이 깊어지고 실제 렌더링 구조를 따라가며 디버깅하기 어려워질 수 있었다. 따라서 현재 문제 규모와 변경 빈도에는 Custom Hook이 가장 균형이 좋다고 판단했다.

Result 정책 변경 시 Hook 내부의 기준만 수정하면 대부분의 화면에 동일하게 반영되도록 개선했다. 그 결과 권한 수정 노력 가능성이 낮아졌고, QA도 역할 기준으로 검증 포인트를 정리할 수 있어 회귀 테스트가 수월해졌다. 프론트엔드 입장에서는 신규 화면을 만들 때도 권한 조건을 새로 설계하기보다 기존 Hook을 조합해 일관된 구조를 유지할 수 있게 되었다.

5. 기술 선택 이유와 설계 의사결정

왜 Custom Hook을 선택했는가

Problem 권한 정책이 변경될 때마다 여러 화면을 동시 수정해야 했고, 일부 누락으로 운영 이슈가 반복되었다. 이 문제를 단순 버그 수정이 아니라 구조적 설계 문제로 봐야 했다.

Requirements 첫째, 권한 정책 변경 시 수정 지점을 최소화해야 했다. 둘째, 화면은 권한 정책의 상세 규칙을 몰라도 사용할 수 있어야 했다. 셋째, 신규 기능 추가 시 같은 권한 구조를 빠르게 재사용할 수 있어야 했다. 넷째, QA와 기획이 확인할 수 있을 정도로 판단 기준이 명확해야 했다.

Alternatives (1) 각 화면에서 조건문을 정리하는 방식은 초기 수정 속도는 빠르지만 장기적으로 또다시 분산된다. (2) HOC는 권한 제어를 컴포넌트 바깥에서 선언적으로 적용할 수 있지만, 여러 권한 규칙이 중첩되면 컴포넌트 계층이 복잡해지고 실제 props 흐름을 추적하기 어렵다. (3) Context API는 권한 상태를 전역으로 공급하는 데는 적합하지만, 이번 요구사항의 핵심은 상태 전파보다 판단 로직의 캡슐화와 재사용이었다.

Decision 권한 판단을 Custom Hook으로 추상화하고, 화면에서는 Hook이 반환하는 결과값만 소비하도록 결정했다. 이렇게 하면 권한 로직은 한 곳에 모이고, 화면은 표현과 흐름 제어에 집중할 수 있다.

Trade-off Custom Hook 역시 만능은 아니다. 너무 많은 정책을 하나의 Hook에 몰아넣으면 내부 복잡도가 커질 수 있고, 권한 판단 범위가 지나치게 넓어지면 오히려 추적이 어려워질 수 있다. 그래서 권한의 공통 기준은 Hook으로 모으되, 화면 특화 예외 케이스는 최소한으로 제한하는 방향을 택했다.

Result 권한 정책 변경 대응 속도가 좋아졌고, 화면별 조건문 중복이 줄었다. 운영 이슈의 재현과 수정도 쉬워졌으며, QA 검증 기준이 역할 중심으로 정리되면서 커뮤니케이션 비용도 낮아졌다.

Learned 공통 로직은 '재사용을 위한 추상화'보다 '변경 비용을 낮추기 위한 추상화'가 더 중요하다는 점을 배웠다. 특히 운영 서비스에서는 처음 구현이 빠른 구조보다, 정책이 바뀌었을 때 어디를 수정해야 하는지가 명확한 구조가 훨씬 가치가 크다.

6. 운영 서비스 관점에서의 실무 포인트

- 권한 문제는 단순 UI 노출 제어가 아니라 운영 정책과 직결되는 영역이기 때문에, 화면별 편의 구현보다 정책 반영 일관성이 더 중요했다.
- 기획 변경은 예외가 아니라 상수라고 보고 구조를 설계해야 했다. 실제로 정책이 여러 차례 바뀌면서 '한 번 구현하고 끝나는 기능'이 아니라는 점이 분명해졌다.
- 유지보수성은 코드 스타일 문제가 아니라 수정 범위 예측 가능성과 회귀 리스크 관리 문제라는 것을 체감했다.
- 협업에서는 프론트엔드가 임의로 권한을 해석하기보다, 역할별 동작 기준을 명시적으로 정리해 기획·QA와 같은 기준으로 소통하는 것이 중요했다.

7. 연관 기능 경험

참여 기능 조직도(Tree), 댓글, 메신저(WebSocket), 화상회의, AI Chat, OCR, STT, WebView Bridge, Camera, Permission 등 협업 플랫폼 성격의 기능과 연계된 화면/흐름을 다루며 프론트엔드 중심으로 유지보수와 기능 개선을 경험했다.

실무 관점 이들 기능은 각각 구현 방식이 다르더라도 실제 운영 단계에서는 권한, 진입 흐름, 예외 처리, 사용자 경험 일관성이라는 공통 과제로 연결되었다. 특히 권한 구조가 정리되지 않으면 기능이 늘어날수록 화면별 분기가 증가해 유지보수성이 급격히 떨어질 수 있다는 점을 경험했다.

8. 면접에서 바로 설명할 수 있는 답변 포인트

가장 어려웠던 버그는 무엇인가요 가장 어려웠던 문제는 권한 정책 변경 이후 일부 화면에서 권한 없는 액션이 노출되던 이슈였습니다. 단일 화면 버그처럼 보였지만 실제로는 권한 판단이 여러 UI에 분산된 구조적 문제였습니다. 그래서 개별 화면 수정이 아니라 권한 판단을 Custom Hook으로 모으는 리팩토링으로 해결했습니다.

운영 중 가장 기억에 남는 경험은 무엇인가요 운영 단계에서 권한 정책이 반복적으로 바뀌었을 때입니다. 초기 구조에서는 화면마다 조건문을 다시 찾아 수정해야 했지만, 리팩토링 이후에는 공통 Hook 수정 중심으로 대응할 수 있어 변경 대응 속도와 안정성이 모두 좋아졌습니다.

왜 Custom Hook 을 선택했나요 이번 문제의 본질은 전역 상태 공유가 아니라 권한 판단 로직의 재사용과 일관성 확보였습니다. Context 는 상태 공급에는 적합하지만 판단 로직을 정리하는 데는 과했고, HOC 는 Wrapper 구조가 깊어질수록 디버깅이 어려울 수 있어 Hook 이 가장 적절했습니다.

리팩토링을 진행한 이유는 무엇인가요 운영 중 변경 비용이 너무 컸기 때문입니다. 기능 하나를 수정할 때마다 여러 화면을 같이 확인해야 했고, 수정 누락이 QA 와 운영 이슈로 이어졌습니다. 중복 제거보다 변경 지점 통합과 회귀 리스크 축소가 리팩토링의 핵심 목적이었습니다.

유지보수성을 어떻게 고려했나요 화면이 권한 정책을 직접 해석하지 않게 만들고, 공통 Hook 이 판단 결과를 제공하도록 구조를 바꿨습니다. 덕분에 정책 변경 시 수정 범위를 예측할 수 있었고, QA 검증 포인트도 역할 기준으로 정리할 수 있었습니다.

다시 개발한다면 무엇을 개선하시겠습니까 초기 단계부터 역할-기능 매핑 기준을 더 명시적으로 문서화하고, 화면 단위가 아니라 역할 시나리오 중심으로 검증 체계를 가져가고 싶습니다. 그래야 기능이 늘어나도 권한 예외가 분산되지 않고, 프론트와 QA, 기획이 같은 기준으로 빠르게 합의할 수 있습니다.